

Embedded Systems and IoT

Ingegneria e Scienze Informatiche - UNIBO

a.a 2025/2026

Lecturer: Prof. Alessandro Ricci

[Lab Notes]

OOP FOR ARDUINO: FROM JAVA TO C++

OUTLINE

- This module provides a brief intro to the C++ programming, starting from Java as baseline (that has been already introduced, studied and used in the OOP course) and discussing the differences
 - we will consider only the subset of features and elements that are actually implemented by the avr-gcc compiler for AVR/Arduino platform

WIRING E TOOLCHAIN ATMEL AVR

- The language adopted to implement Arduino program, on which the Wiring framework is based, is a subset of the C++ language
 - reference tools for programming Atmel AVR microcontrollers: the avr-gcc compiler, avr-libc library, avr-binutils tool
 - <https://gcc.gnu.org/wiki/avr-gcc>
 - <http://www.nongnu.org/avr-libc/>
- Wiring/avr-gcc implements a relevant subset of the C++ language
 - some features are not supported because of the restrictions in terms of memory available on microcontrollers
 - support available or not specified through compiler options
- Among the key supported/implemented features:
 - inheritance, polymorphism and virtual functions, template/generics
- Among the features *not* supported
 - exceptions

THE C++ OOP LANGUAGE

- **The C++ Language**
 - introduced in early 80ies by Bjarne Stroustrup as an OO extension of the C language
 - reference OO language for programming “performance-critic” applications and system-oriented applications
- Latest standard standard: C++20 (2020)
 - <https://isocpp.org/std/the-standard>
 - <https://it.wikipedia.org/wiki/C%2B%2B>
 - rich library support
 - in particular: standard template library STL
- A main language considered when Java was designed
 - strong similarities
 - ..but C++ is more complex and low-level

BJARNE STROUSTRUP



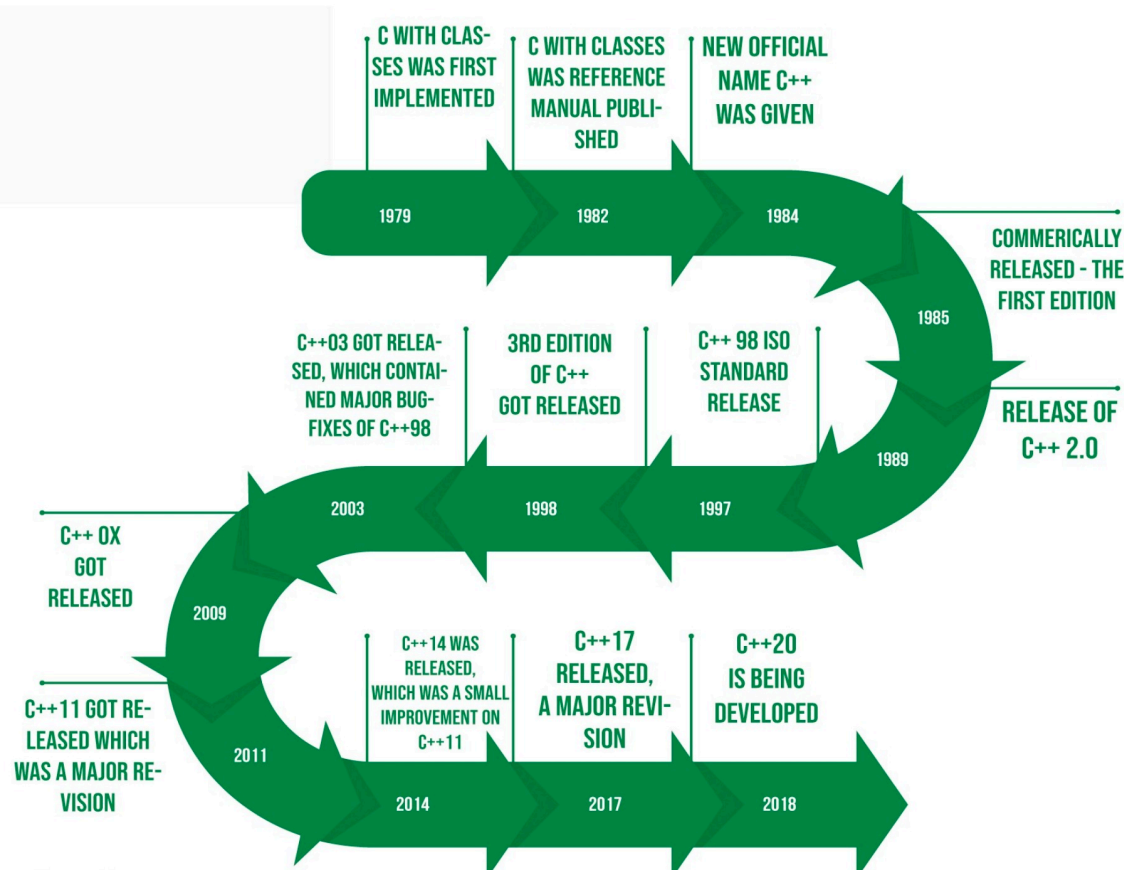
(all'inizio degli anni 80)



(2020)

HISTORY OF C++

History of C++



[from: <https://www.geeksforgeeks.org/history-of-c/>]

C++ VERSUS JAVA

- Key points (compared to C and Java)
 - **no** runtime architecture / virtual machine
 - manual memory management - new, delete
 - no garbage collector, yes pointers
 - destructors, besides constructors
 - both dynamic and static objects
 - multiple inheritance
 - no interfaces, yes abstract classes
 - polymorphism based on functions (methods) *virtual*
 - operator overloading
 - passing arguments by value and by reference
 - source organisation not bound to the file system (like in C)
 - namespaces instead of packages
 - ...
- Good tour on C++ features : <https://isocpp.org/tour>

CLASS DEFINITION

```
class ClassName {  
public:  
    <public constructors  
    + destructors  
    + member functions>  
protected:  
    <protected constructors  
    + destructors  
    + member functions>  
private:  
    <private attributes>  
    <private constructors  
    + destructors  
    + member functions>  
};
```

```
class Counter {  
  
public:  
    Counter(int value){  
        count = value;  
    }  
  
    void inc() {  
        count++;  
    }  
  
    int getValue(){  
        return count;  
    }  
  
private:  
    int count;  
};
```


WITH HEADER/IMPLEMENTATION SEPARATION

```
class Counter {  
public:  
    Counter(int value);  
    void inc();  
    int getValue();  
private:  
    int count;  
};
```

Counter.hpp or Counter.h

```
#include "Counter.hpp"  
  
Counter::Counter(int value){  
    count = value;  
}  
  
void Counter::inc() {  
    count++;  
}  
  
int Counter::getValue(){  
    return count;  
}
```

Counter.cpp

OO TERMINOLOGY IN C++ VS. JAVA

- Methods in C++ are called *member functions*
- Fields are called *attributes*

STATIC AND DYNAMIC OBJECTS

- Static objects - allocated in the global memory or on the stack

```
Counter c(10);  
...  
c.inc();
```

- Dynamic objects - allocated on the heap

```
Counter* c1;  
...  
c1 = new Counter(20);  
...  
c1->inc();  
...  
delete c1;
```

NOTA:
explicit deallocation with
delete

CONSTRUCTORS AND DESTRUCTORS

- Like in Java, we have **constructors**
 - called when an object is instantiated, either statically or dynamically with **new**
 - useful for initialising the object
 - can be overloaded
 - differently from Java: specific syntax to initialise attributes
- Differently from Java here we have also **destructors**
 - syntax: `~ClassName()`
 - called before object deallocation
 - both for static and dynamic objects

```
class C
{
public:
    C(T1 a, T2 b, T3 c);
    ~C();
    ...
private:
    T1 x; T2 y;
    T3* z;
}

C::C(T1 a, T2 b, T3 c): x(a), y(b)
{
    z = new C1(...);
}

C::~~C()
{
    delete z;
}
```

INHERITANCE AND POLYMORPHISM

- Support for Multiple inheritance and abstract classes
- Virtual member functions for enabling polymorphism

```
class Instrument {  
public:  
    virtual void play() = 0;  
protected:  
    void makeSound(String s){  
        Serial.println(s);  
    }  
};
```

```
class Percussion: public Instrument {  
public:  
    void play(){  
        makeSound("toc");  
    }  
};
```

```
class Piano: public Instrument {  
public:  
    void play(){  
        makeSound("ding");  
    }  
};
```

INHERITANCE AND POLYMORPHISM

```
Instrument* list[2];

void setup(){
    Serial.begin(9600);
    list[0] = new Piano();
    list[1] = new Percussion();
}

void loop(){
    for (int i = 0; i < 2; i++){
        list[i]->play();
    }
    delay(1000);
}
```

OPERATOR OVERLOADING

- In C++ it is possible to override also operators — both unary and binary operators
 - member function syntax
- Esempio

```
class Complex
{
    public:
        Complex(double re,double im)
            :real(re),imag(im)
            {};
        Complex operator+(const Complex& other);
        Complex operator=(const Complex& other);
    private:
        double real;
        double imag;
};

Complex Complex::operator+(const Complex& other)
{
    double result_real = real + other.real;
    double result_imaginary = imag + other.imag;
    return Complex( result_real, result_imaginary );
}
```

```
...
Complex a(1.2,1.3);
Complex b(2.1,3);
Complex c = a + b;
```

REFERENCE VARIABLES

- In C++ we can have *reference variables*, as alias to existing variables

```
int i = 5;  
int& j = i; // reference/alias of i  
...  
j = 8; // updates the content of i
```

- reference variables must be always initialised when declared

- Argument passing by reference
 - by specifying reference variables as parameters, it is like passing the pointer to that variable => the function can change the value of the variable referenced by the parameter

```
void swap(int& x, int& y){ int c = x; x = y; y = c; }  
...  
int a = 4; int b = 6;  
swap(a,b); // after the call: a = 6 and b = 4
```


TEMPLATE

- Support for generics (class, methods)
 - parametric polymorphism

```
template<class T>
class Holder {
    T* elem;

public:
    Holder(T* el){
        elem = el;
    }

    T* get(){
        return elem;
    }
};
```

```
...
Counter* c1 = new Counter(20);
...
Holder<Counter> holder(c1);
...
```